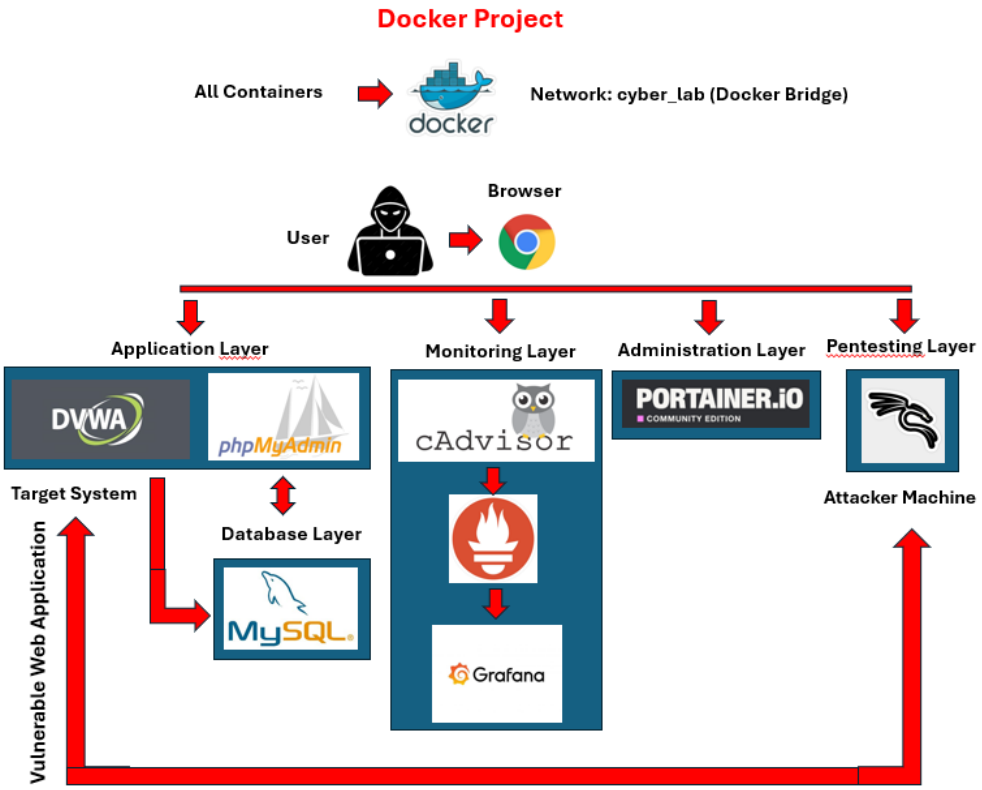
LABORATOIRE DE CYBERSÉCURITÉ

Docker • Kali Linux • DVWA • MySQL • Grafana • Prometheus • Portainer

TP : Mise en place d'un laboratoire de cybersécurité avec Docker



Plan

1. Introduction

2. Objectifs

3. Outils et logiciels utilisés

4. Principe du TP

5. Architecture du projet

6. Réalisation
6.1 Création du dossier projet
6.2 Création du fichier docker-compose.yml
6.3 Lancement de l'infrastructure
6.4 Accès au conteneur Kali Linux
6.5 Accès à l'application DVWA
6.6 Accès à phpMyAdmin
6.7 Accès à cAdvisor
6.8 Accès à Grafana
6.9 Accès à Prometheus
6.10 Accès à Portainer
6.11 L'Arrête des conteneurs

7. Résultats obtenus

8. Analyse

9. Conclusion

1. Introduction

La conteneurisation représente aujourd'hui une solution moderne pour déployer rapidement des applications dans des environnements isolés, légers et performants.

Grâce à Docker, il est possible de créer plusieurs services indépendants tout en facilitant leur administration, leur portabilité et leur maintenance.

Dans ce TP, nous avons réalisé un laboratoire local de cybersécurité composé de plusieurs conteneurs spécialisés.

2.Objectifs

Les objectifs de ce TP sont :

- Déployer une infrastructure multi-conteneurs avec Docker Compose
- Héberger une application vulnérable (DVWA)
- Intégrer une base de données MySQL
- Superviser les services avec Grafana et Prometheus
- Administrer les conteneurs via Portainer
- Simuler un environnement professionnel de test

3. Outils et logiciels utilisés

Outil	Description
Docker Desktop	Plateforme de conteneurisation
Visual Studio Code	Éditeur de configuration
Invite de commandes (CMD)	Exécution des commandes
Kali Linux	Outils de cybersécurité
DVWA	Application web vulnérable
MySQL	Base de données
phpMyAdmin	Administration MySQL
Portainer	Gestion Docker
Grafana	Visualisation
Prometheus	Monitoring
cAdvisor	Collecte métriques

4. Principe du TP

Le principe du TP consiste à mettre en place un laboratoire virtuel basé sur Docker Compose.

Chaque conteneur possède un rôle spécifique :

- DVWA : cible de test
- Kali Linux : machine d'analyse
- MySQL : stockage des données
- phpMyAdmin : administration base de données
- Portainer : gestion des conteneurs
- cAdvisor : collecte des métriques
- Prometheus : stockage des métriques
- Grafana : tableaux de bord

Tous les services communiquent via un réseau Docker privé nommé : cyber_lab.

5. Architecture du projet

```
Utilisateur (Navigateur Web)
        |
        |------ localhost:8080 --> DVWA
        |------ localhost:8081 --> phpMyAdmin
        |------ localhost:3000 --> Grafana
        |------ localhost:8000 --> Portainer
        |------ localhost:9090 --> Prometheus
        |------ localhost:8082 --> cAdvisor
```

Tous les conteneurs utilisent le réseau interne : cyber_lab.

6. Réalisation

6.1 Création du dossier projet

Exécuter les commandes suivantes :

```
cd Documents
mkdir docker-cyber-lab
cd docker-cyber-lab
```

Explication :
- cd : changer de dossier
- mkdir : créer un nouveau dossier

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sadki>cd Documents

C:\Users\sadki\Documents>mkdir docker-cyber-lab

C:\Users\sadki\Documents>cd docker-cyber-lab

C:\Users\sadki\Documents\docker-cyber-lab>
```

6.2 Création du fichier docker-compose.yml

Créer le fichier principal contenant la configuration de l'infrastructure.

Ce fichier permet de définir :

- les services
- les ports
- les volumes
- les réseaux
- les dépendances

Le fichier docker-compose.yml centralise tout le projet dans un seul document.

docker-compose.yml

C: > Users > sadki > Documents > docker-cyber-lab > docker-compose.yml > { } services > { } mysql > abc image

Run All Services | docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container plat

```yaml
services:

  Run Service
  dvwa:
    image: vulnerables/web-dvwa
    container_name: dvwa
    ports:
      - "8080:80"
    restart: unless-stopped
    depends_on:
      - mysql
    networks:
      - cyber_lab

  Run Service
  mysql:
    image: mysql:5.7
    container_name: mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: password
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - cyber_lab
```

```
▷Run Service
phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: phpmyadmin
  restart: unless-stopped
  depends_on:
    - mysql
  environment:
    PMA_HOST: mysql
    MYSQL_ROOT_PASSWORD: root
  ports:
    - "8081:80"
  networks:
    - cyber_lab

▷Run Service
kali:
  image: kalilinux/kali-rolling
  container_name: kali
  tty: true
  stdin_open: true
  restart: unless-stopped
  mem_limit: 1g
  cpus: 1
  networks:
    - cyber_lab

```

```
▷Run Service
portainer:
  image: portainer/portainer-ce
  container_name: portainer
  restart: unless-stopped
  ports:
    - "8000:9000"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - portainer_data:/data
  networks:
    - cyber_lab

▷Run Service
cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: cadvisor
  restart: unless-stopped
  ports:
    - "8082:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  networks:
    - cyber_lab

```

```
▷Run Service
  grafana:
    image: grafana/grafana
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - cyber_lab
▷Run Service
  prometheus:
    image: prom/prometheus
    container_name: prometheus
    restart: unless-stopped
    ports:
    - "9090:9090"
    volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
    networks:
    - cyber_lab

networks:
  cyber_lab:
    driver: bridge

volumes:
  mysql_data:
  portainer_data:
  grafana_data:
```

Création du nouveau fichier « prometheus.yml » avec VS Code. Dans le même chemin " C:\Users\sadki\Documents\docker-cyber-lab "

```
! prometheus.yml ×
C: > Users > sadki > Documents > docker-cyber-lab > ! prometheus.yml > ...
prometheus.json - Prometheus configuration file (prometheus.json)
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

Le fichier docker-compose.yml et prometheus.yml centralise la configuration de toute l'infrastructure et permet de lancer l'ensemble des services avec une seule commande :

docker compose up -d

Ce TP permet ainsi d'appliquer concrètement les notions suivantes :

- conteneurisation
- orchestration de services
- réseaux virtuels
- persistance des données
- supervision système
- environnement de test sécurisé

En résumé, ce laboratoire reproduit une mini architecture professionnelle utilisée dans les environnements DevOps et cybersécurité modernes.

6.3 Lancement de l'infrastructure

Commande utilisée :

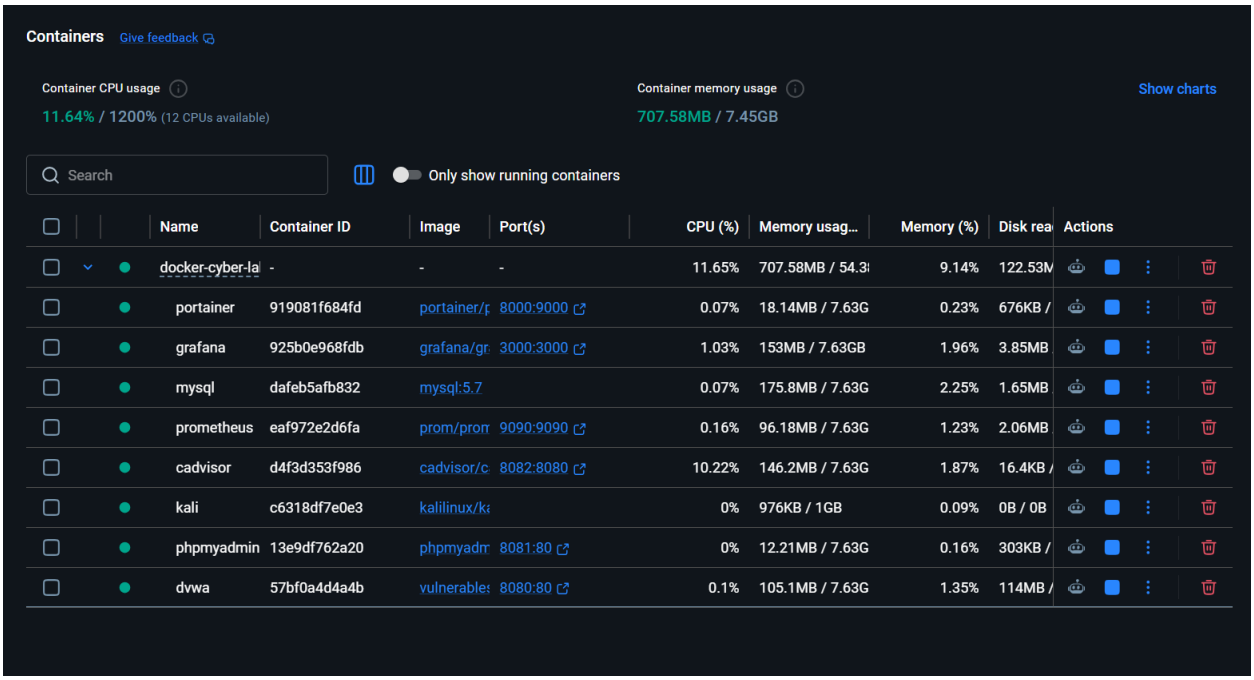
```
docker-compose up -d
```

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sadki\Documents\docker-cyber-lab>docker-compose up -d
[+] up 9/9
 ✔Network docker-cyber-lab_cyber_lab Created
 ✔Container grafana                  Started
 ✔Container mysql                    Started
 ✔Container prometheus               Started
 ✔Container cadvisor                 Started
 ✔Container portainer                Started
 ✔Container kali                     Started
 ✔Container phpmyadmin               Started
 ✔Container dvwa                     Started
```

Explication :

- up : démarrer les services
- -d : exécution en arrière-plan

Nous retournons dans Docker pour vérifier si les conteneurs ont été installés.



Vérification : docker ps

Cette commande affiche tous les conteneurs actifs.

```
docker ps
```

```
C:\Users\sadki\Documents\docker-cyber-lab>docker ps
CONTAINER ID   IMAGE                              COMMAND                  CREATED              STATUS                  PORTS
           NAMES
13e9df762a20   phpmyadmin/phpmyadmin              "/docker-entrypoint.…"   About a minute ago   Up 2 minutes            0.0.0.0:8081->80/tcp, [::]:8081->80/
tcp        phpmyadmin
57bf0a4d4a4b   vulnerables/web-dvwa               "/main.sh"               About a minute ago   Up 2 minutes            0.0.0.0:8080->80/tcp, [::]:8080->80/
tcp        dvwa
919081f684fd   portainer/portainer-ce             "/portainer"             About a minute ago   Up 2 minutes            0.0.0.0:8000->9000/tcp, [::]:8000->9
000/tcp    portainer
925b0e968fdb   grafana/grafana                    "/run.sh"                About a minute ago   Up 2 minutes            0.0.0.0:3000->3000/tcp, [::]:3000->3
000/tcp    grafana
dafeb5afb832   mysql:5.7                          "docker-entrypoint.s…"   About a minute ago   Up 2 minutes            3306/tcp, 33060/tcp
           mysql
eaf972e2d6fa   prom/prometheus                    "/bin/prometheus --c…"   About a minute ago   Up 2 minutes            0.0.0.0:9090->9090/tcp, [::]:9090->9
090/tcp    prometheus
d4f3d353f986   gcr.io/cadvisor/cadvisor:latest    "/usr/bin/cadvisor -…"   About a minute ago   Up 2 minutes (healthy)  0.0.0.0:8082->8080/tcp, [::]:8082->8
080/tcp    cadvisor
c6318df7e0e3   kalilinux/kali-rolling             "bash"                   About a minute ago   Up 2 minutes
           kali
```

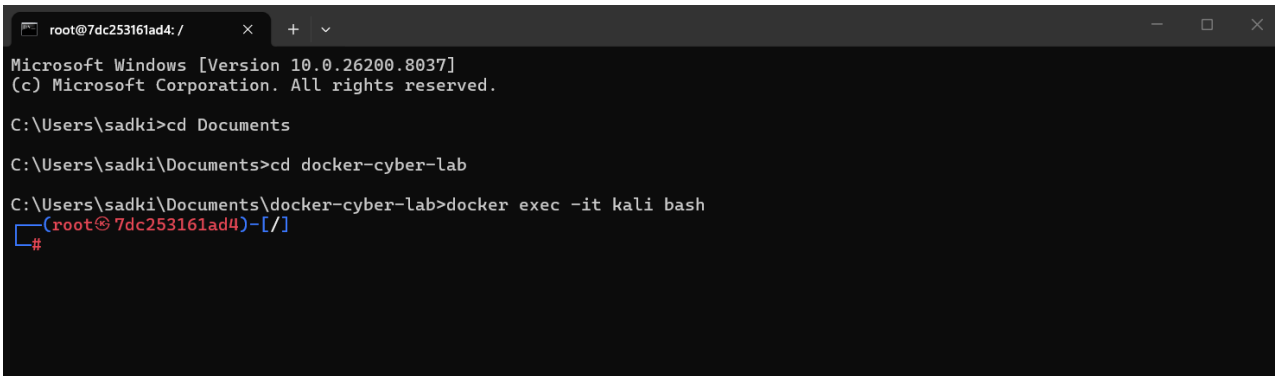
Accès aux services

Outils :	Link :	Identifiants :
DVWA	http://localhost:8080	Username : admin Password : password
phpMyAdmin	http://localhost:8081	Username : root Password : root
Grafana	http://localhost:3000	Username : admin Password : admin
Prometheus	http://localhost:9090	----------
Portainer	http://localhost:8000	Username : admin1234 Password : admin1234
cAdvisor	http://localhost:8082	----------

6.4 Accès au conteneur Kali Linux

Commande : `docker exec -it kali bash`

Explication :
Cette commande ouvre un terminal interactif à l'intérieur du conteneur Kali Linux.
Le conteneur Kali permet d'utiliser des outils d'analyse et de test dans un environnement local autorisé.



Mettez à jour votre distribution Kali Linux pour la première fois afin d'obtenir tous les nouveaux accès.

```
┌──(root㉿c6318df7e0e3)-[/]
└─# apt-get update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [15.3 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:5 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Fetched 21.4 MB in 4s (6124 kB/s)
Reading package lists... Done
```

6.5 Accès à DVWA

Dans votre navigateur " Google chrome "

http://localhost:8080

Les informations de connexion sont fournies par défaut.

Username : admin

Password : password

1. Intégration de MySQL

```
▷Run Service
mysql:
  image: mysql:5.7
  container_name: mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: dvwa
    MYSQL_USER: dvwa
    MYSQL_PASSWORD: password
  volumes:
    - mysql_data:/var/lib/mysql
  networks:
    - cyber_lab
```

Définition :

MySQL est un système de gestion de base de données relationnelle (SGBD).

Rôle dans le projet :

L'ajout de MySQL permet de stocker les données des applications de manière structurée et sécurisée.

Avantages :

- stockage organisé des données.
- gestion multi-utilisateurs.
- performance élevée.
- compatibilité avec plusieurs applications web.

Port utilise : 3306

6.6 Accès à phpMyAdmin

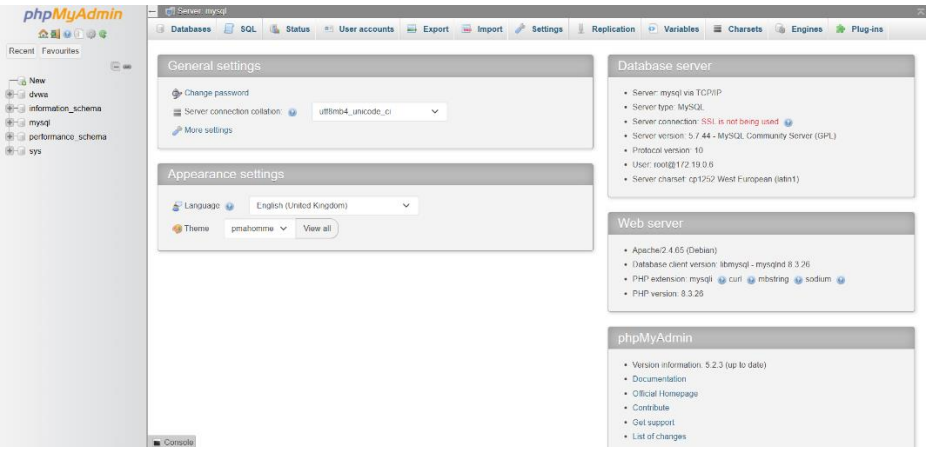
Dans votre navigateur " Google chrome "

http://localhost:8081/

Les informations de connexion sont fournies par défaut.

Username : root

Password : root



2.Intégration de phpMyAdmin

```
▷Run Service
phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: phpmyadmin
  restart: unless-stopped
  depends_on:
    - mysql
  environment:
    PMA_HOST: mysql
    MYSQL_ROOT_PASSWORD: root
  ports:
    - "8081:80"
  networks:
    - cyber_lab
```

Définition :

phpMyAdmin est une interface web permettant l'administration de MySQL via navigateur.

Rôle dans le projet :

Il facilite la gestion de la base de données sans utiliser uniquement les commandes SQL.

Fonctions principales :

- création de bases de données
- création de tables
- import/export
- exécution de requêtes SQL
- gestion des utilisateurs

Port utilisé : 8080 (selon configuration)

6.7 Accès à cAdvisor

Partie Theorique

Définition :

cAdvisor signifie :

Container Advisor

Outil développé pour analyser l'utilisation des ressources des containers Docker.

Importance :

Il fournit les métriques suivantes :

- CPU usage
- mémoire RAM
- trafic réseau
- espace disque
- statistiques par container

Dans ce projet :

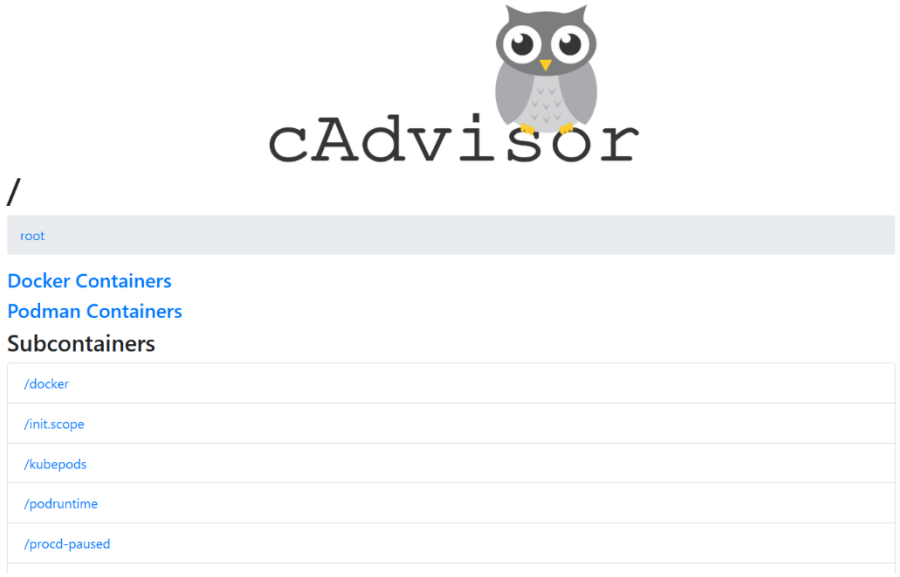
Il observe :

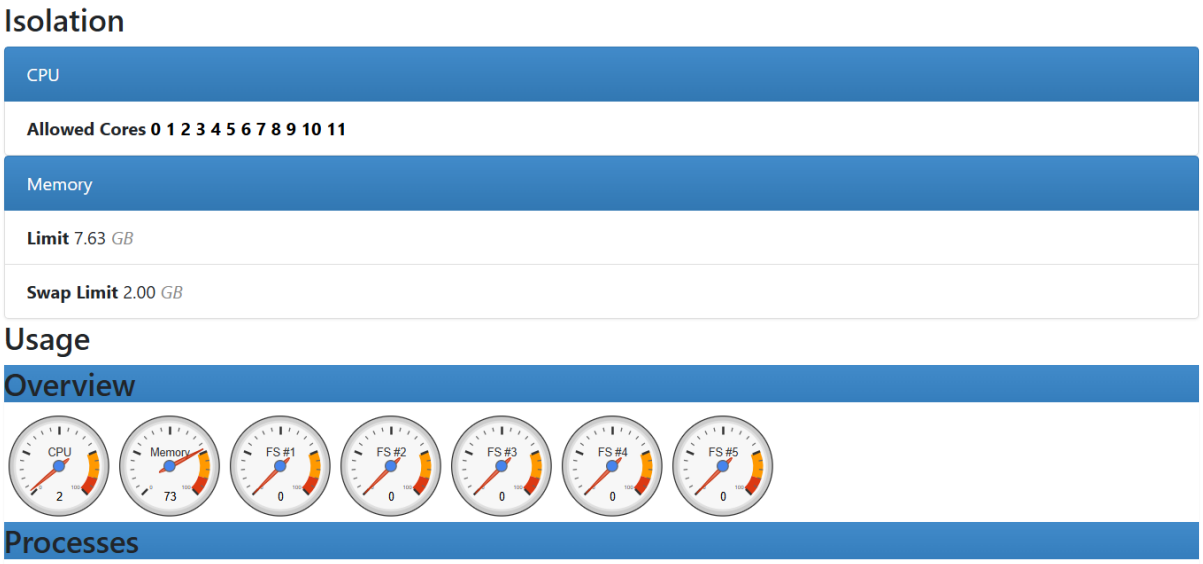
- DVWA
- MySQL
- Kali
- Grafana
- Portainer

Partie Pratique

Dans votre navigateur " Google chrome "

http://localhost:8082/containers/





6.8 Accès à Grafana
Partie Theorique

Définition :

Grafana est une plateforme open source de **visualisation et supervision** permettant d'afficher des données provenant de différentes sources (Prometheus, MySQL, InfluxDB, etc.).

Importance :

- tableaux de bord professionnels
- visualisation claire des performances
- analyse en temps réel
- historique des données
- alertes intelligentes
- aide à la prise de décision

Dans ce projet :

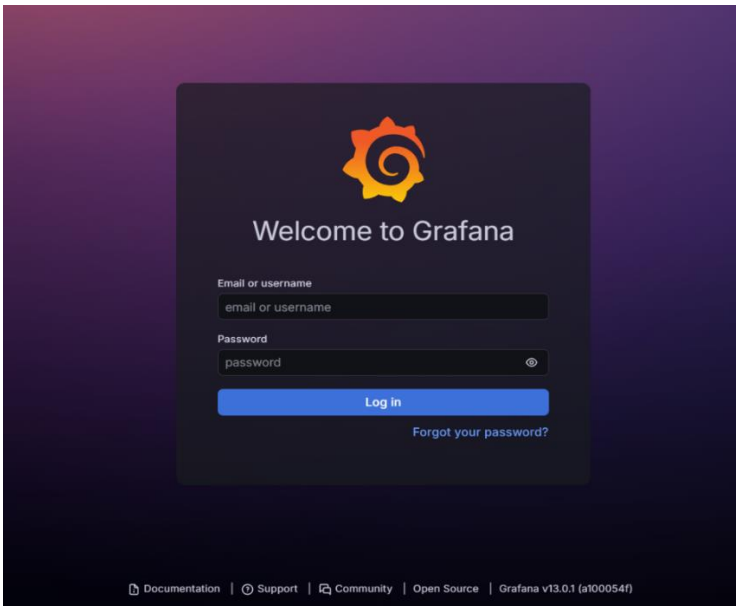
Grafana permet de surveiller :

- CPU des containers
- RAM
- réseau
- état des services

Partie Pratique

Dans votre navigateur " Google chrome "

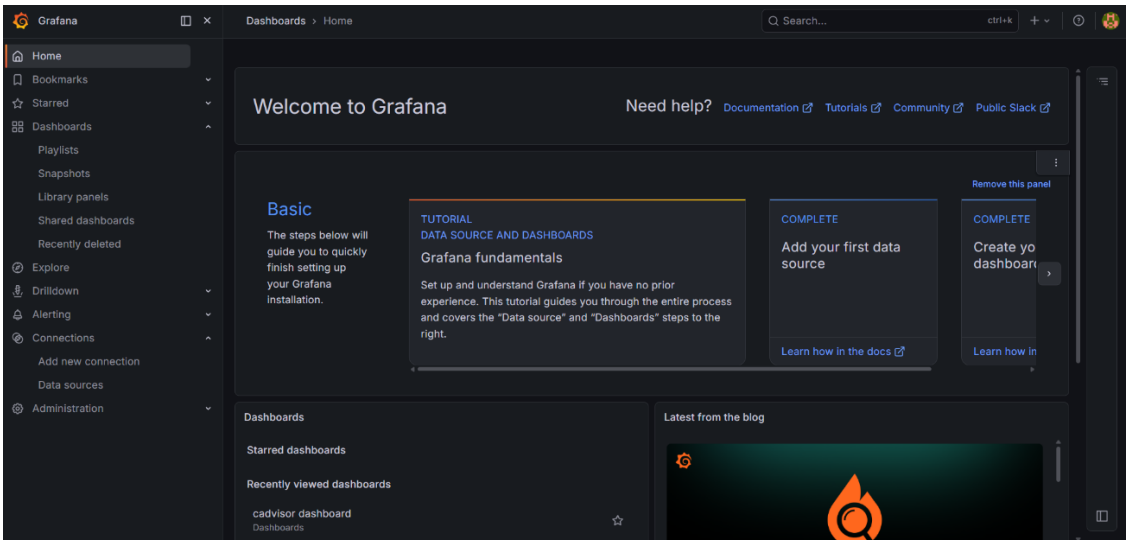
http://localhost:3000/login



Les informations de connexion sont fournies par défaut.

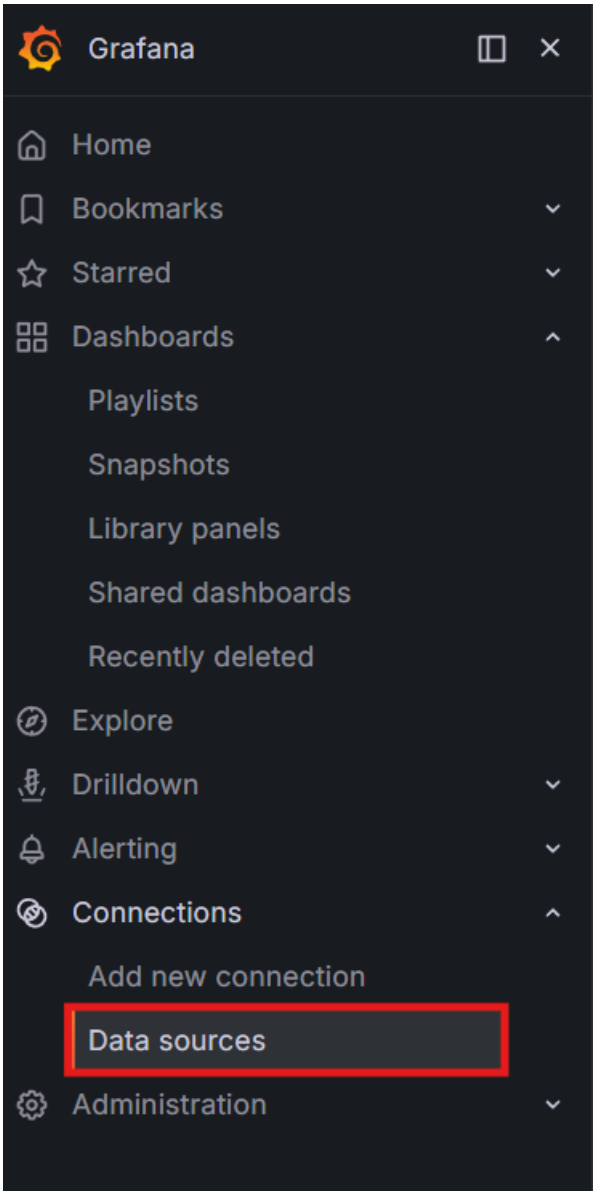
Username : admin

Password : admin

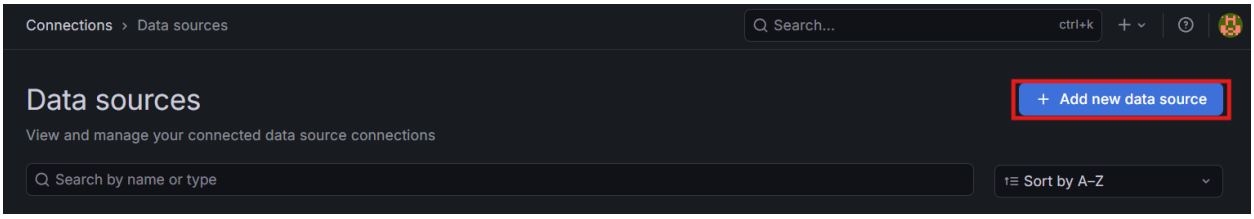


Pour lier Grafana avec Prometheus

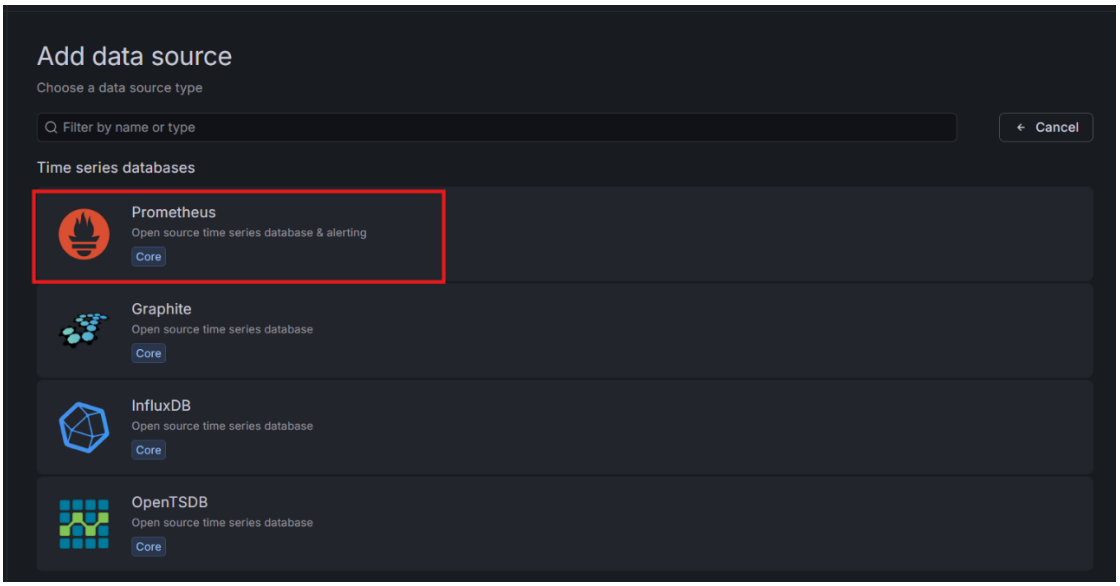
1. choisir Data sources



2. cliquer sur Add new data sources

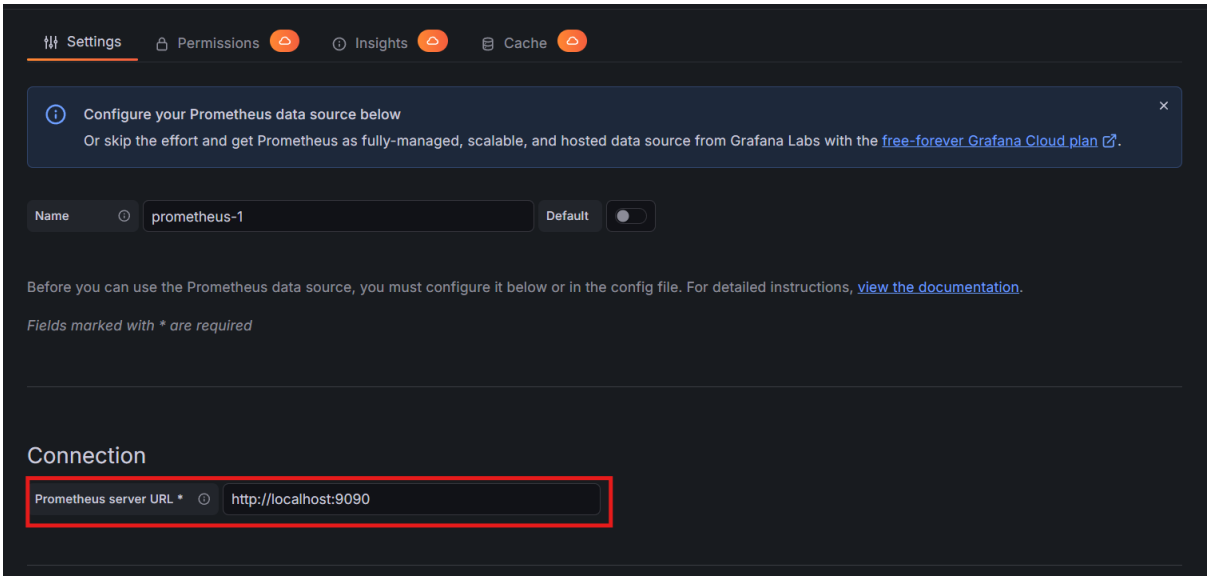


3. choisir Prometheus

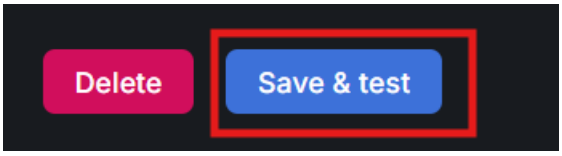


Dans la partie Connection, donner pour le page web du Prometheus cette URL

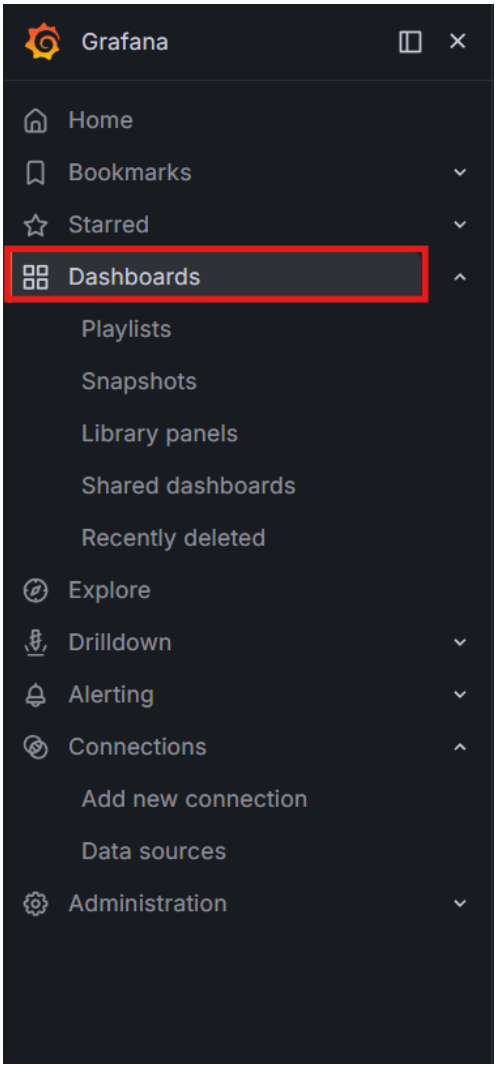
http://localhost:9090



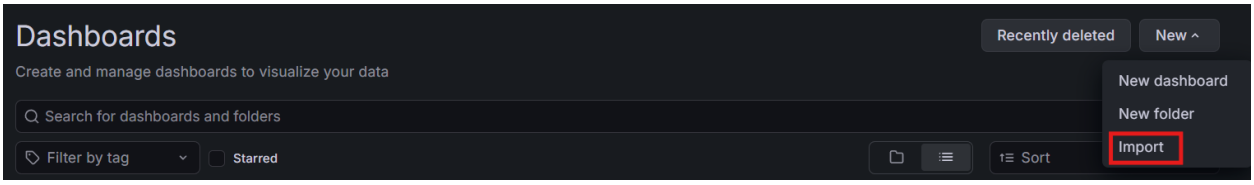
Enregistre la configuration d'apres la button « Save & test »



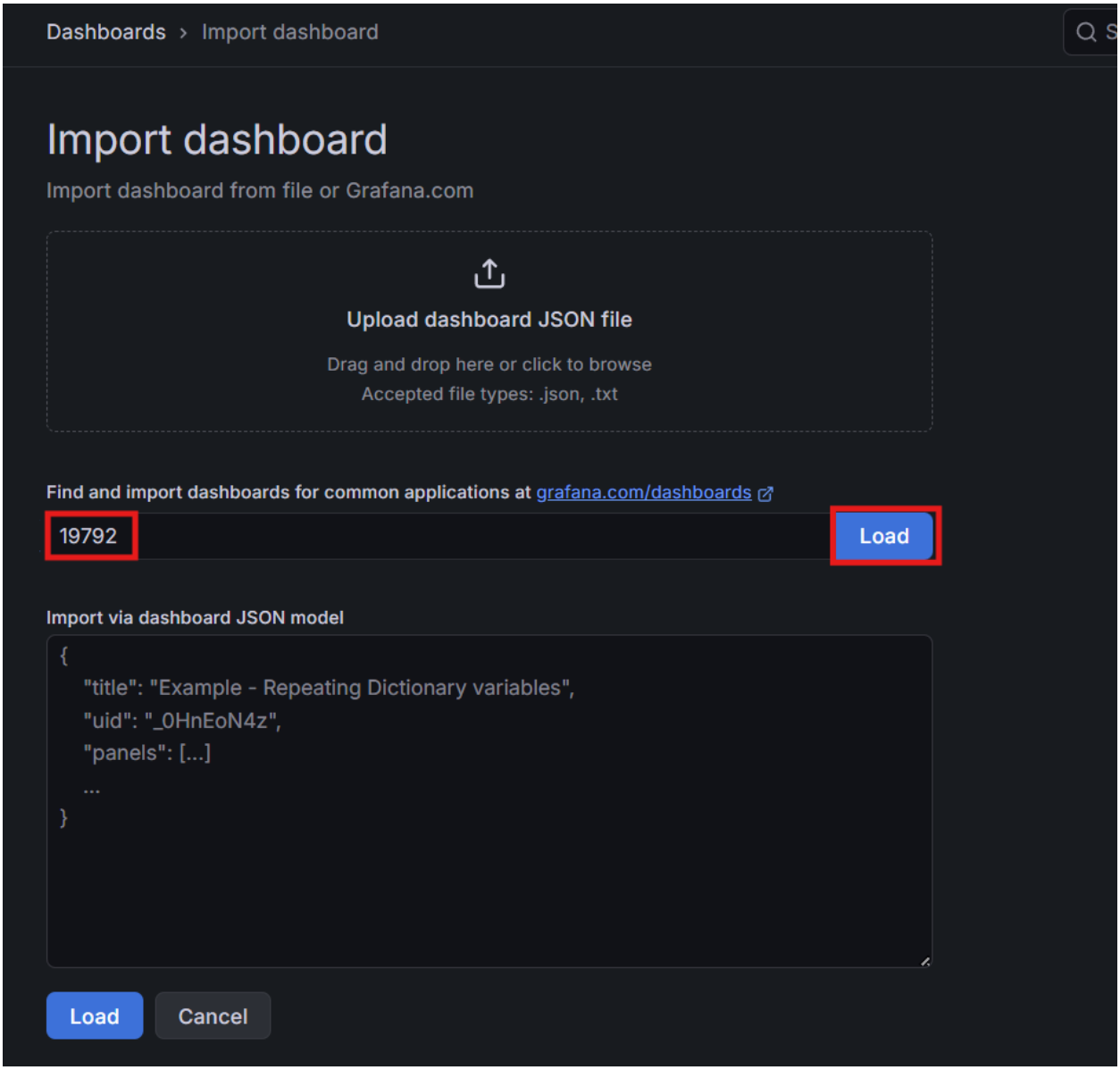
4. retourner au Dashboards et choisir New en click sur Import



choisir New et click sur Import



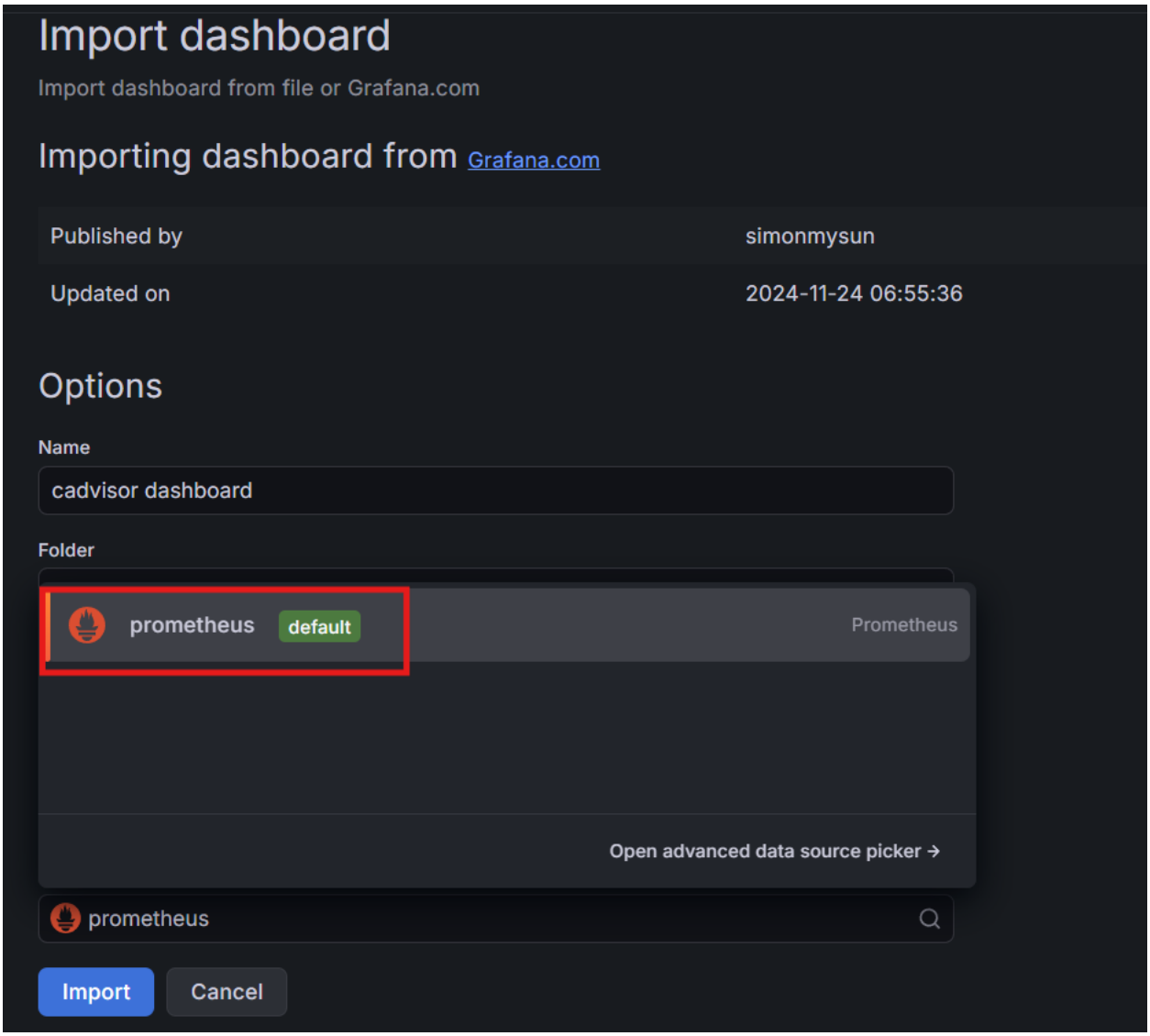
Taper 19792 et click sur Load pour actualise l'etap



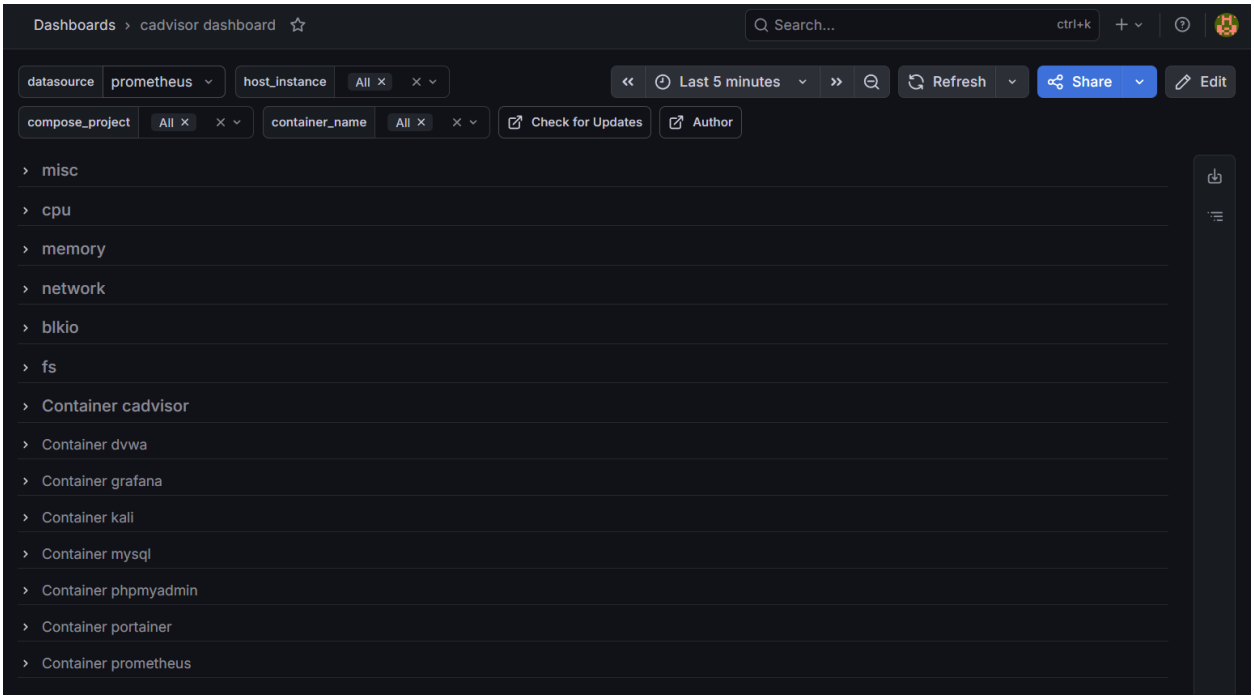
19792 est un ID de Dashboard Grafana prêt à l'emploi (template prédéfini) utilisé pour la supervision des environnements Docker / containers via Prometheus.

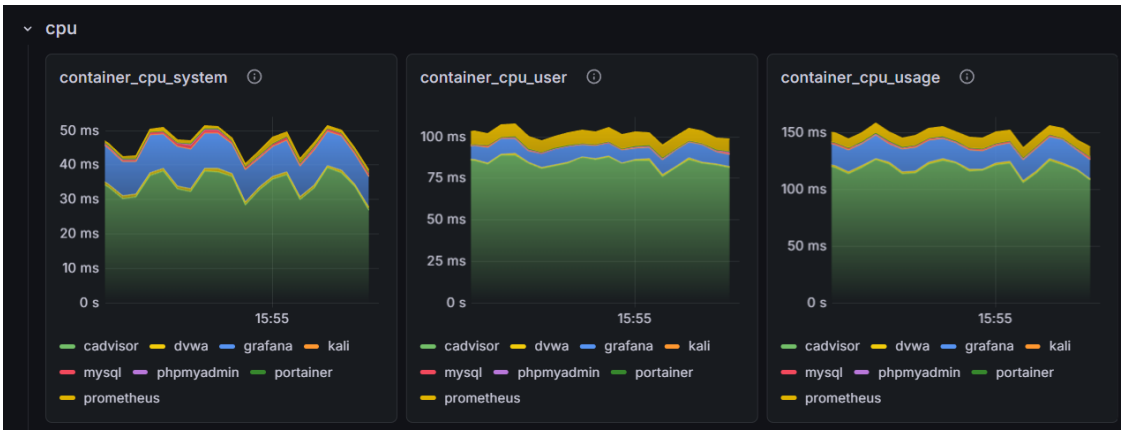
Dans folder selectionne promethuse qu en a choisir précédemment dans l'etape de add data source

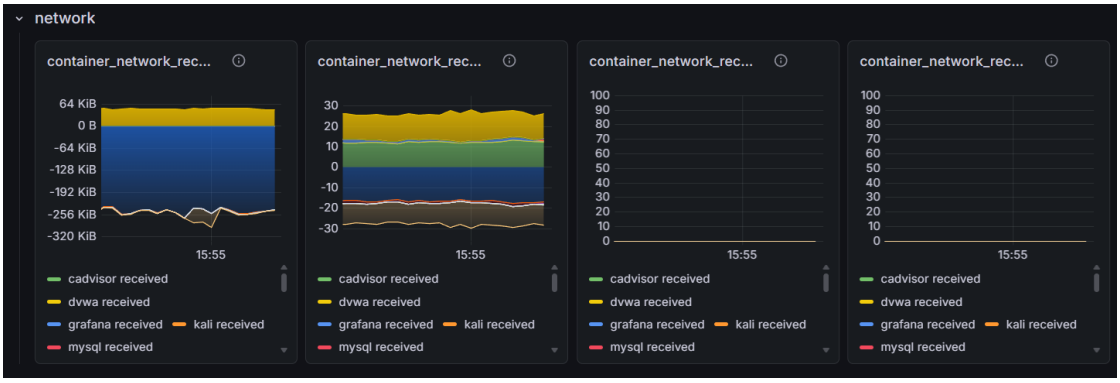
Et click sur import



5. voir et analyse la supervision de l'tat du docker et ces composentes







6.9 Accès à Prometheus

Partie Theorique

Définition :

Prometheus est un système open source de **monitoring et collecte de métriques** développé initialement par SoundCloud.

Il récupère régulièrement les données depuis les services surveillés.

Importance :

- collecte automatique des métriques
- stockage des séries temporelles
- requêtes puissantes (PromQL)
- surveillance continue
- intégration avec Grafana

Dans ce projet :

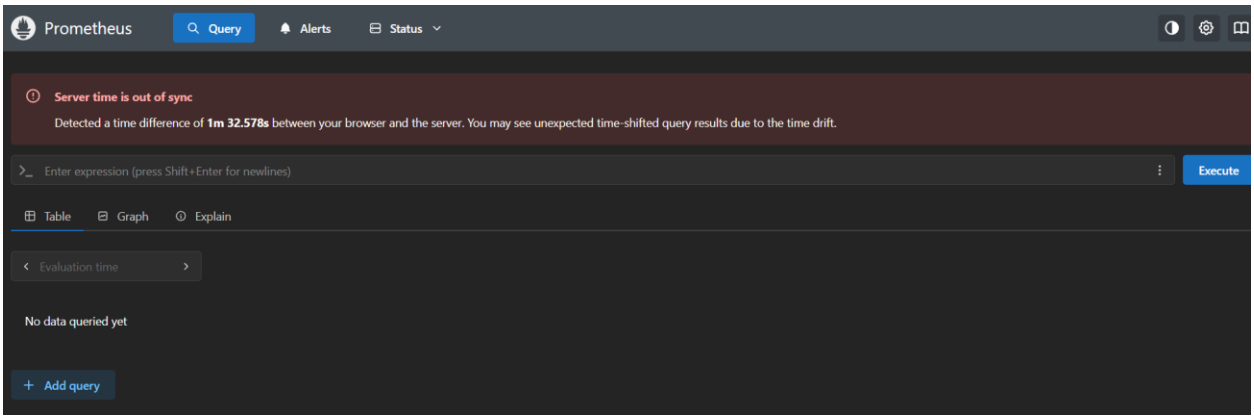
Prometheus récupère les données depuis :

cAdvisor

Partie Pratique

Dans votre navigateur " Google chrome "

http://localhost:9090/



6.10 Accès à Portainer

Partie Theorique

Définition :

Portainer est une interface graphique de gestion Docker.

Importance :

Il facilite l'administration des containers sans utiliser uniquement la ligne de commande.

Fonctions :

- créer containers
- supprimer containers
- démarrer / arrêter
- voir logs
- gérer images
- gérer volumes
- gérer réseaux

Dans ce projet :

Portainer sert à administrer toute l'infrastructure Docker.

Avantage :

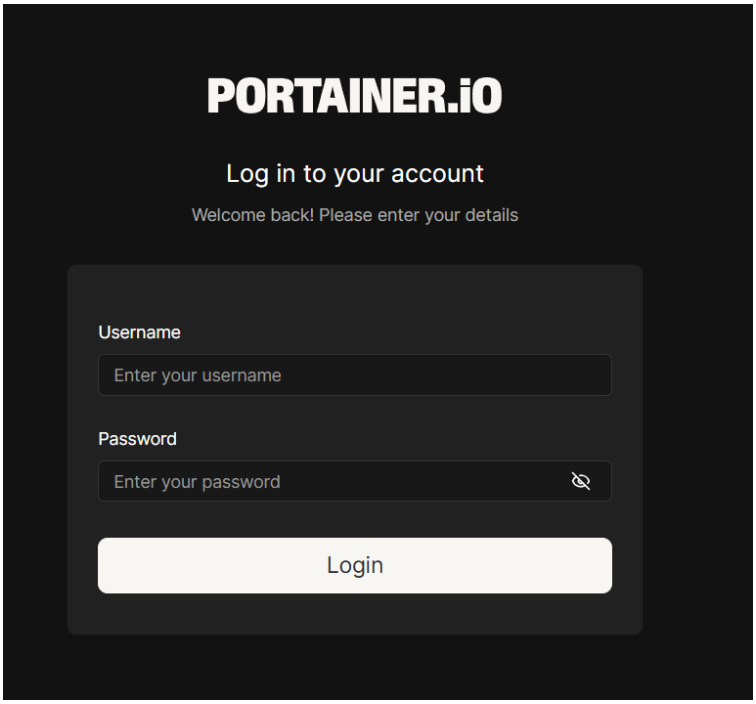
Administration simplifiée sans dépendre uniquement du terminal Linux.

Port utilisé :9000

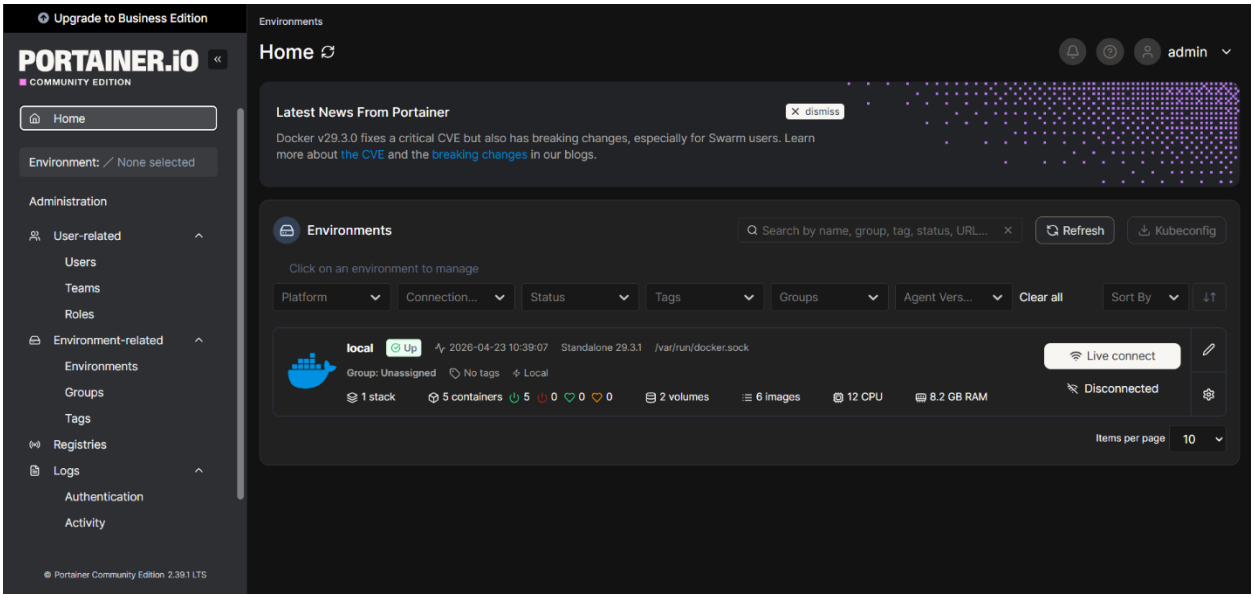
Partie Pratique

Dans votre navigateur " Google chrome "

http://localhost:8000/



Connectez-vous avec vos informations et enregistrez-les



3.Intégration de PORTAINER.IO

```
▷Run Service
portainer:
  image: portainer/portainer-ce
  container_name: portainer
  restart: unless-stopped
  ports:
    - "8000:9000"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - portainer_data:/data
  networks:
    - cyber_lab
```

Quelle est la différence entre tous ces superviseurs ?

Outil	Rôle principal	Type
Portainer	Gestion des containers Docker	Administration
cAdvisor	Collecte métriques containers	Monitoring technique
Prometheus	Stockage + collecte métriques	Backend monitoring
Grafana	Visualisation des données	Dashboard

Résumé simple

Portainer : Gérer Docker facilement

cAdvisor : Observer les performances des containers

Prometheus : Enregistrer les statistiques

Grafana : Afficher les statistiques visuellement

Relation entre tous ces superviseurs

Supervision du laboratoire
Le système de supervision repose sur trois composants :

- cAdvisor : collecte les métriques des conteneurs
- Prometheus : stocke les données collectées
- Grafana : affiche les tableaux de bord graphiques

Chaîne de supervision :

Containers --> cAdvisor --> Prometheus --> Grafana

Conclusion

Ces outils sont complémentaires et non concurrents. Ensemble, ils forment une solution complète de supervision et d'administration moderne pour les infrastructures Docker.

4.Intégration des Persistent Volumes

Définition :

```
volumes:
  mysql_data:
  portainer_data:
```

Les volumes Docker permettent de conserver les données même après suppression ou redémarrage des containers.

Rôle dans le projet :

Assurer la persistance des données critiques comme :

- données MySQL
- configuration Portainer
- fichiers applicatifs

Importance :

Sans volumes, les données peuvent être perdues après recréation du container.

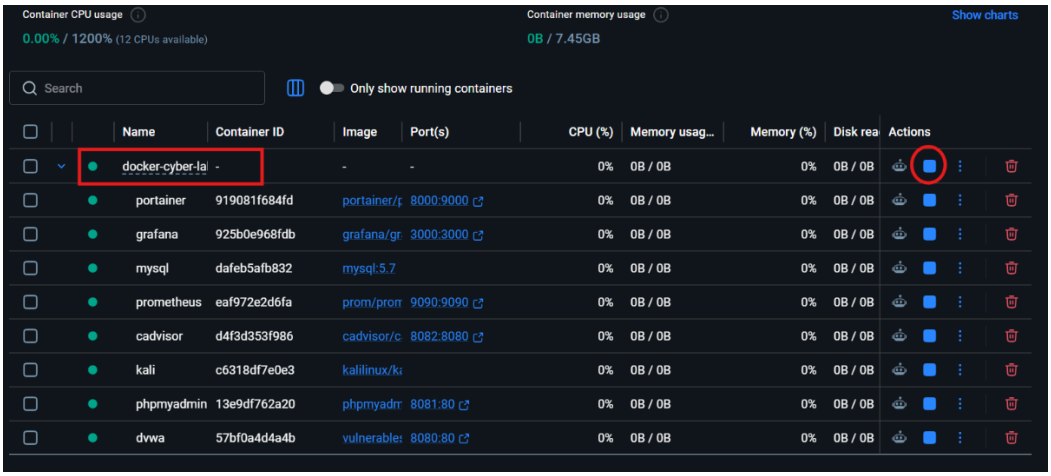
6.11 L'Arrête des conteneurs

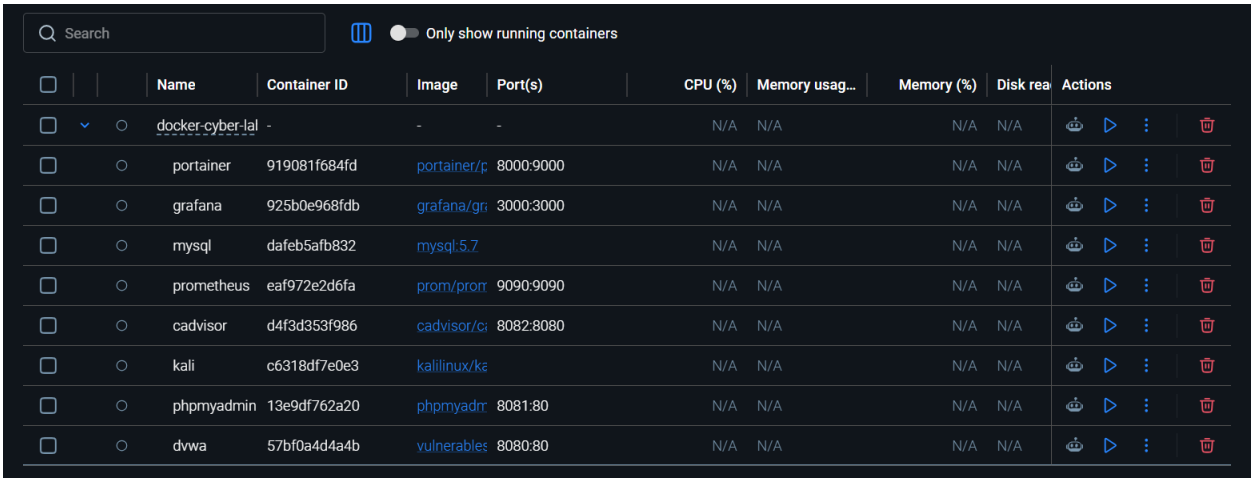
Arrêtez les conteneurs au point d'entrée de la commande "cmd "

```
docker-compose  stop
```

```
C:\Users\sadki\Documents\docker-cyber-lab>docker-compose  stop
[+] stop 8/8
 ✔Container grafana    Stopped
 ✔Container kali       Stopped
 ✔Container portainer  Stopped
 ✔Container phpmyadmin Stopped
 ✔Container prometheus Stopped
 ✔Container dvwa       Stopped
 ✔Container cadvisor   Stopped
 ✔Container mysql      Stopped
```

Ou via l'interface graphique, c'est-à-dire l'application. " docker desktop "





► : Pour lancer les conteneurs

■ : Pour arrêter les conteneurs

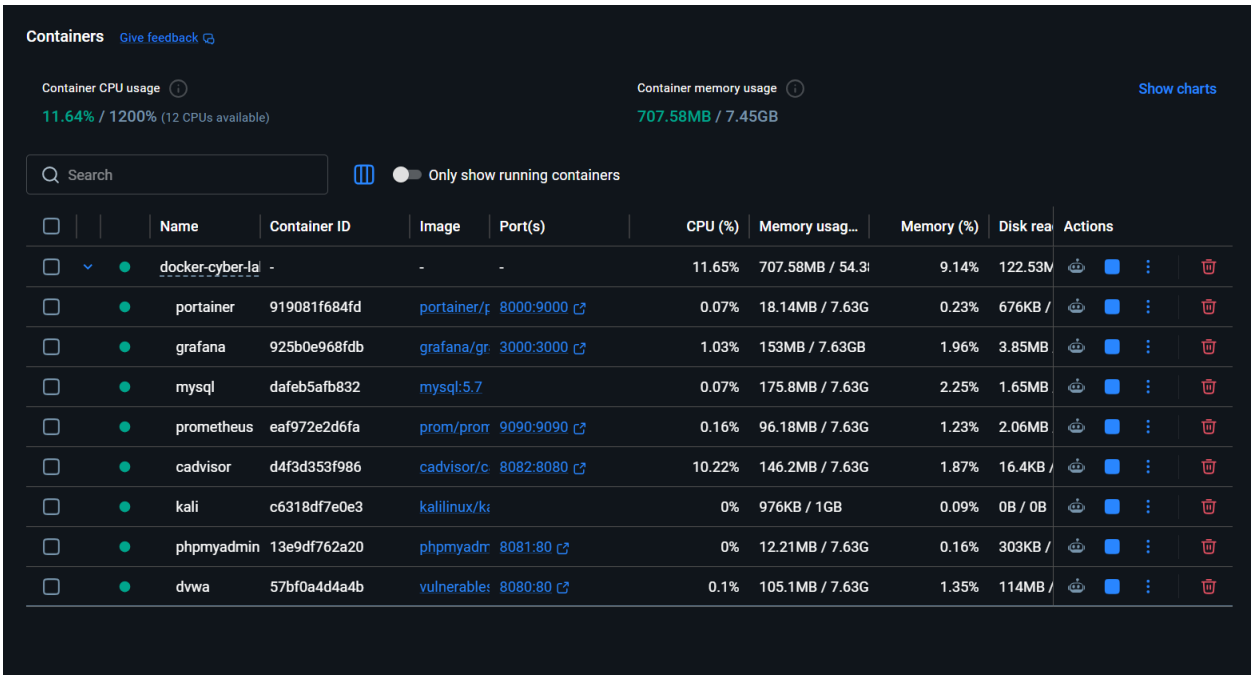
Lancer les conteneurs au point d'entrée de la commande "cmd "

```
docker-compose start
```

```
C:\Users\sadki\Documents\docker-cyber-lab>docker-compose start
[+] start 8/8
 ✔Container mysql      Started
 ✔Container prometheus Started
 ✔Container portainer  Started
 ✔Container cadvisor   Started
 ✔Container grafana    Started
 ✔Container kali       Started
 ✔Container phpmyadmin Started
 ✔Container dvwa       Started
```

7. Résultat

Le laboratoire fonctionne correctement avec huit conteneurs actifs et connectés.

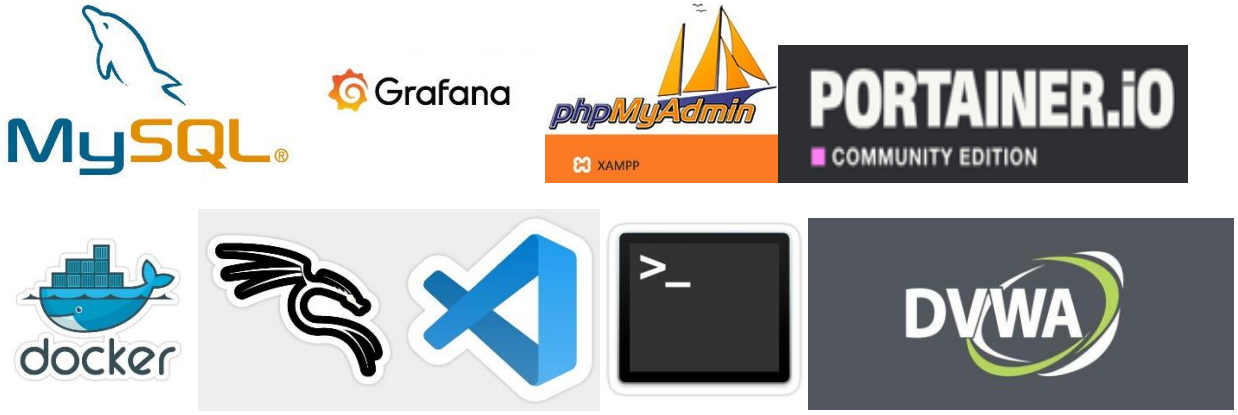


8. Analyse

Docker présente plusieurs avantages :

- Rapidité de déploiement
- Faible consommation de ressources
- Isolation des services
- Portabilité
- Facilité d’administration
- Architecture professionnelle moderne

Ce laboratoire démontre l’intérêt de Docker dans les environnements DevOps et cybersécurité.



Conclusion Finale

Ce TP m'a permis de comprendre concrètement les principes de la conteneurisation ainsi que la mise en place d'une infrastructure multi-services moderne.

J'ai pu déployer, administrer et superviser plusieurs conteneurs dans un environnement local sécurisé.

Ce projet représente une base solide pour les architectures professionnelles orientées cloud, DevOps et cybersécurité.